

BAZY DANYCH – PROJEKT – DOKUMENTACJA

TEMAT: FIRMA TURYSTYCZNA

2025/2026 IPpp SEM4 – sekcja 1

SKŁAD SEKCJI:

Kacper Skowroński,

Krzysztof Bierzanek,

Michał Walterowski,

Igor Juraszek

SPIS TREŚCI

Specyfikacja Wymagań.....	2
Specyfikacja Scenariuszy i Architektury.....	6
Specyfikacja UML.....	10
Instrukcja Obsługi Strony.....	11
Specyfikacja Zewnętrzna Systemu	16
Specyfikacja Wewnętrzna Systemu	18
Podsumowanie Realizacji Projektu	23
Zmiany w Bazie Danych	25

SPECYFIKACJA WYMAGAŃ

Wymagania Funkcjonalne

1. Zarządzanie bazą ofert wakacyjnych

- **Dodawanie ofert:** System umożliwia dodanie nowej oferty do bazy.
- **Wymagane dane:** System wymaga podania i zapisuje dla każdej oferty: miejscowość, nazwę hotelu, cenę (w wybranej walucie) oraz rodzaj wyżywienia.
- **Modyfikacja:** System umożliwia modyfikację wszystkich parametrów istniejących ofert.
- **Usuwanie i archiwizacja:** System umożliwia usunięcie oferty lub jej archiwizację (ukrycie przed klientami).
- **Status dostępności:** System pozwala na przypisanie statusu dostępności oferty (np. dostępna, wyprzedana).
- **Integracja z API:** System automatycznie pobiera i wyświetla na karcie oferty aktualne dane pogodowe (temperatura, opis warunków) dla wybranej miejscowości, wykorzystując zewnętrzne API.

2. Zarządzanie bazą klientów

- **Rejestracja:** System umożliwia rejestrację nowego klienta.
- **Dane klienta:** System wymaga podania i zapisuje następujące dane: imię, nazwisko, pełny adres zamieszkania, adres e-mail.
- **Weryfikacja:** System automatycznie weryfikuje poprawność formatu wprowadzanego adresu e-mail.
- **Edycja danych:** System umożliwia edycję i aktualizację danych klienta.
- **Usuwanie profilu:** System umożliwia całkowite usunięcie profilu klienta z bazy danych.

3. Moduł wyszukiwania i filtrowania

- **Przeglądanie:** System umożliwia klientom przeglądanie pełnej listy dostępnych ofert wakacyjnych.
- **Wyszukiwanie:** System umożliwia wyszukiwanie ofert na podstawie wprowadzonej nazwy miejscowości.
- **Filtrowanie zaawansowane:** System umożliwia filtrowanie ofert po:
 - Nazwie konkretnego hotelu.
 - Rodzaju wyżywienia.
 - Zadany przedział cenowy.
 - Dacie planowanych wakacji.
 - Ilości osób.
- **Sortowanie:** System umożliwia sortowanie wyników wyszukiwania według ceny (rosnąco lub malejąco).

4. Obsługa rezerwacji

- **Tworzenie rezerwacji:** System umożliwia przypisanie wybranego klienta do konkretnej oferty wakacyjnej w celu utworzenia rezerwacji.
- **Zarządzanie statusami:** System umożliwia pracownikom biura zmianę statusu rezerwacji (np. nowa, opłacona, w trakcie realizacji, anulowana).
- **Historia:** System przechowuje pełną historię rezerwacji przypisanych do danego klienta.

5. Moduł raportowania i analityki

- **Raport sprzedaży:** System generuje raport sumarycznej sprzedaży (liczby i wartości rezerwacji) w zadanym przedziale czasowym.
- **Zestawienia popularności:** System generuje zestawienie najpopularniejszych miejscowości wybieranych przez klientów oraz najpopularniejszych hoteli i rodzajów wyżywienia.
- **Lista dedykowana:** System generuje listę klientów, którzy dokonali rezerwacji wyjazdu do konkretnej miejscowości.
- **Eksport danych:** System umożliwia eksport wygenerowanych raportów do pliku zewnętrznego (np. **PDF, CSV, XLSX**).

Wymagania Niefunkcjonalne

1. Wydajność i pojemność

- **Czas odpowiedzi:** Czas odpowiedzi systemu podczas wyszukiwania i filtrowania ofert nie może przekroczyć **2 sekund** przy obciążeniu do kilkuset jednoczesnych użytkowników.
- **Generowanie raportów:** Czas generowania raportów przekrojowych nie powinien przekraczać **5 sekund**.
- **Skalowalność bazy:** Baza danych musi być przystosowana do przechowywania i płynnego przetwarzania kilku tysięcy rekordów klientów i ofert.

2. Bezpieczeństwo i uprawnienia

- **Uwierzytelnianie:** System wymaga uwierzytelnienia (logowania) dla pracowników biura korzystających z paneli administracyjnych i modułu raportowania.
- **Kontrola dostępu (RBAC):** Poziomy dostępu muszą być oparte na rolach (np. klient widzi tylko oferty, pracownik ma dostęp do bazy klientów, administrator do całego systemu).
- **Ochrona haseł:** Hasła użytkowników w bazie danych muszą być **jednokierunkowo zaszyfrowane**.
- **Automatyczne wylogowanie:** System automatycznie wylogowuje użytkownika administracyjnego po **30 minutach bezczynności**.

3. Zgodność z prawem i ochrona danych (RODO)

- **Zgody marketingowe:** System wymaga zaznaczenia przez klienta zgody na przetwarzanie danych osobowych podczas rejestracji lub dokonywania rezerwacji.

- **Prawo do bycia zapomnianym:** System umożliwia trwałą i nieodwracalną anonimizację danych osobowych na żądanie klienta.
- **Audyt dostępu:** System rejestruje historię dostępu do danych osobowych (logowanie aktywności pracowników).

4. Użyteczność (UX/UI)

- **Responsywność:** Interfejs użytkownika musi być w pełni responsywny (**RWD**), gwarantując poprawne działanie na komputerach, tabletach i smartfonach.
- **Intuicyjność:** Obsługa systemu raportowania musi być intuicyjna i nie wymagać od pracowników znajomości języków zapytań bazodanowych (np. SQL).
- **Obsługa błędów:** Komunikaty o błędach podczas wypełniania formularzy muszą w jasny sposób wskazywać pole wymagające poprawy.

5. Niezawodność i środowisko operacyjne

- **Kopie zapasowe:** System musi automatycznie wykonywać pełną kopię zapasową bazy danych (backup) **co 24 godziny**.
- **Dostępność (SLA):** Gwarantowana dostępność systemu wynosi **99,9%** w skali miesiąca.
- **Kompatybilność:** System musi funkcjonować poprawnie w najnowszych, stabilnych wersjach przeglądarek: Google Chrome, Mozilla Firefox, Safari oraz Microsoft Edge.

Analiza MoSCoW

M – Must have (Wymagania krytyczne)

Absolutnie niezbędne funkcje, bez których system nie może działać i nie spełnia podstawowego celu biznesowego. Projekt musi je zawierać w pierwszej wersji.

- Moduł dodawania, edycji i usuwania ofert (z polami: miejscowość, hotel, cena, wyżywienie).
- Formularz rejestracji klientów pobierający imię, nazwisko, adres zamieszkania i e-mail.
- Funkcjonalność przypisywania klientów do ofert w celu utworzenia rezerwacji.
- Wyszukiwarka ofert pozwalająca filtrować po miejscowości, hotelu, wyżywieniu i cenie.
- Podstawowy moduł raportowania generujący listę rezerwacji i zestawienia popularności miejscowości.
- Bezpieczne logowanie oparte na rolach dla pracowników biura i administratorów.
- Szyfrowanie haseł w bazie danych.
- Zgodność z RODO, w tym zbieranie zgód na przetwarzanie danych i realizacja prawa do bycia zapomnianym.
- Baza danych zdolna do płynnego przetwarzania tysięcy rekordów.
- Responsywny interfejs użytkownika (RWD) działający na urządzeniach mobilnych i desktopowych.

S – Should have (Wymagania ważne)

Funkcje o wysokim prioryecie, które znacząco poprawiają wartość systemu i ułatwiają pracę, ale ich brak nie uniemożliwia startu platformy. Można je wdrożyć w kolejnym etapie.

- Sortowanie wyników wyszukiwania ofert według ceny (rosnąco/malejąco).
- Edycja i samodzielna aktualizacja danych profilowych przez zalogowanego klienta.
- Zarządzanie statusami dostępności oferty (np. wyprzedana, ostatnie miejsca).
- Raporty sumarycznej sprzedaży z określonych przedziałów czasowych.
- Funkcja eksportu raportów biznesowych do plików PDF, CSV lub XLSX.
- Automatyczne wylogowywanie pracowników po 30 minutach bezczynności.
- Pełna automatyzacja codziennych kopii zapasowych bazy danych (backup).
- Rejestrowanie historii logowań i dostępu do danych osobowych przez pracowników.
- Integracja z zewnętrznym API pogodowym.

C – Could have (Wymagania przydatne)

Funkcjonalności typu „nice-to-have”. Zwiększają komfort użytkowania, ale mają niższy priorytet. Ich realizacja zależy od czasu i budżetu.

- Automatyczne powiadomienia e-mail wysyłane do klientów po zmianie statusu rezerwacji.
- Wizualizacja danych w module raportowania za pomocą wykresów kołowych i słupkowych.
- Walidacja adresów e-mail w czasie rzeczywistym podczas wypełniania formularza rejestracyjnego.
- Śledzenie historii własnych rezerwacji z poziomu panelu klienta.
- Komunikaty o błędach w formularzach dynamicznie podświetlające konkretne, błędnie wypełnione pole.

W – Won't have (Wymagania pominięte w tej wersji)

Funkcje świadomie wykluczone z aktualnego zakresu prac, zaplanowane do ewentualnego wdrożenia w przyszłości.

- Integracja z bramkami płatności online (np. PayU, BLIK, PayPal) do automatycznego opłacania wycieczek.
- System opinii i ocen w skali od 1 do 5 wystawianych hotelom przez klientów po powrocie.
- Integracja zewnętrznych API do rezerwacji biletów lotniczych (np. Skyscanner).
- Dedykowana, natywna aplikacja mobilna do pobrania z App Store lub Google Play.
- Zaawansowany moduł sztucznej inteligencji rekomendujący oferty na podstawie historii wyszukiwań użytkownika.

SPECYFIKACJA SCENARIUSZY I ARCHITEKTURY

Architektura Systemu: Aktorzy i Role

- **Klient:** Przegląda oferty, rejestruje się, dokonuje rezerwacji i zarządza swoim profilem.
- **Pracownik biura:** Zarządza bazą ofert, zmienia statusy rezerwacji i generuje raporty.
- **Administrator:** Odpowiada za zarządzanie uprawnieniami, bezpieczeństwo haseł i kopie zapasowe.
- **Użytkownik niezalogowany:** Przegląda oferty, tworzy konto/loguje się.

Diagram Przypadków Użycia

(W tym miejscu w strukturze dokumentu znajduje się sekcja dedykowana dla diagramu przypadków użycia).

Scenariusz (A): Dokonanie Rezerwacji przez Klienta

Wymagania wstępne

- Klient jest zarejestrowany i zalogowany do systemu.
- Wybrana oferta posiada status "**dostępna**".
- System jest responsywny (działa poprawnie na urządzeniu klienta).

Scenariusz główny

1. Klient wyszukuje ofertę, wpisując miejscowość w wyszukiwarce.
2. Klient wybiera konkretną ofertę z listy wyników. **(A.2)**
3. System wyświetla formularz rezerwacyjny z danymi klienta (imię, nazwisko, adres, e-mail). **(A.3)**
4. Klient zaznacza zgodę na przetwarzanie danych osobowych (RODO). **(A.1)**
5. Klient klika przycisk "**Potwierdź rezerwację**".
6. System weryfikuje dane i tworzy rezerwację.

Scenariusze alternatywne

- **SCENARIUSZ (A.1):** Klient nie zaznaczył zgody RODO. System wyświetla komunikat o błędzie i blokuje możliwość rezerwacji.
- **SCENARIUSZ (A.2):** W trakcie rezerwacji oferta zmieniła status na "**wyprzedana**". System informuje o braku miejsc i proponuje powrót do wyszukiwarki.
- **SCENARIUSZ (A.3):** Wprowadzony adres e-mail ma błędny format. System podświetla pole i wymaga poprawy.

Efekt

- W bazie danych powstaje nowa rezerwacja ze statusem "**Nowa**".
- Rezerwacja zostaje dopisana do historii klienta.
- Dostępność oferty zostaje automatycznie zaktualizowana.

Scenariusz (B): Zarządzanie Ofertą przez Pracownika

Wymagania wstępne

- Pracownik jest zalogowany do panelu administracyjnego.
- Pracownik posiada rolę z uprawnieniami do edycji bazy ofert.

Scenariusz główny

1. Pracownik wybiera funkcję **"Dodaj nową ofertę"**.
2. System wyświetla formularz z polami: miejscowość, hotel, cena, waluta, rodzaj wyżywienia. **(B.1)**
3. Pracownik uzupełnia dane i ustawia status oferty na **"dostępna"**.
4. Pracownik klika **"Zapisz"**. **(B.2)**
5. System potwierdza dodanie oferty do bazy danych.

Scenariusze alternatywne

- **SCENARIUSZ (B.1):** Pracownik nie wypełnił jednego z wymaganych pól (np. ceny). System wyświetla komunikat o braku danych.
- **SCENARIUSZ (B.2):** Pracownik jest nieaktywny przez 30 minut. System automatycznie go wyloguje, a niezapisane zmiany zostają utracone.

Efekt

- Nowa oferta staje się widoczna dla klientów w module wyszukiwania.

Scenariusz (C): Generowanie Raportu Sprzedaży

Wymagania wstępne

- Pracownik/Administrator jest zalogowany.
- W systemie znajdują się archiwalne rezerwacje.

Scenariusz główny

1. Pracownik przechodzi do modułu raportowania.
2. Wybiera typ raportu: **"Zestawienie najpopularniejszych miejscowości"**.
3. Ustawia przedział czasowy i klika **"Generuj"**. **(C.1)**
4. System przetwarza dane i wyświetla zestawienie. **(C.2)**
5. Pracownik klika **"Eksportuj"** i wybiera format **PDF** lub **XLSX**.

Scenariusze alternatywne

- **SCENARIUSZ (C.1):** Brak rezerwacji w wybranym okresie. System wyświetla pusty raport z komunikatem informacyjnym.
- **SCENARIUSZ (C.2):** Błąd serwera podczas generowania pliku. System informuje o problemie technicznym.

Efekt

- Plik z raportem zostaje pobrany na dysk lokalny użytkownika.
- Aktywność pracownika (dostęp do danych rezerwacji) zostaje zapisana w logach aktywności.

Scenariusz (D): Przeglądanie i Wyszukiwanie Ofert (Gość)

Wymagania wstępne

- Dostęp do Internetu i przeglądarki (Chrome, Firefox, Safari lub Edge).
- System musi być responsywny (**RWD**), aby Gość mógł wygodnie przeglądać oferty na telefonie.

Scenariusz główny

1. Użytkownik wchodzi na stronę główną biura podróży.
2. Wpisuje w wyszukiwarkę nazwę miejscowości (np. "Antalya"). (**D.2**)
3. Ustawia filtry: rodzaj wyżywienia (np. All Inclusive) oraz przedział cenowy. (**D.1**)
4. Sortuje wyniki według ceny rosnąco, aby znaleźć najtańszą opcję.
5. System wyświetla listę dostępnych ofert.

Scenariusze alternatywne

- **SCENARIUSZ (D.1):** Brak ofert spełniających kryteria. System wyświetla komunikat "**Brak ofert**" i sugeruje zmianę filtrów.
- **SCENARIUSZ (D.2):** Użytkownik wpisuje nieistniejącą miejscowość. System informuje o braku wyników dla danej frazy.

Efekt

- Gość znajduje interesującą go wycieczkę i może przejść do kroku rezerwacji lub rejestracji.

Scenariusz (E): Rejestracja Nowego Użytkownika

Wymagania wstępne

- Użytkownik nie posiada jeszcze konta w systemie.

Scenariusz główny

1. Gość klika przycisk "**Zarejestruj się**".
2. Wypełnia formularz, podając: imię, nazwisko, pełny adres zamieszkania oraz adres e-mail. (**E.1**)
3. Użytkownik zaznacza obowiązkową zgodę na przetwarzanie danych osobowych (RODO). (**E.2**)
4. System w czasie rzeczywistym weryfikuje poprawność formatu adresu e-mail.

5. Po kliknięciu **"Utwórz konto"**, system zapisuje dane w bazie, a hasło zostaje jednokierunkowo zaszyfrowane.

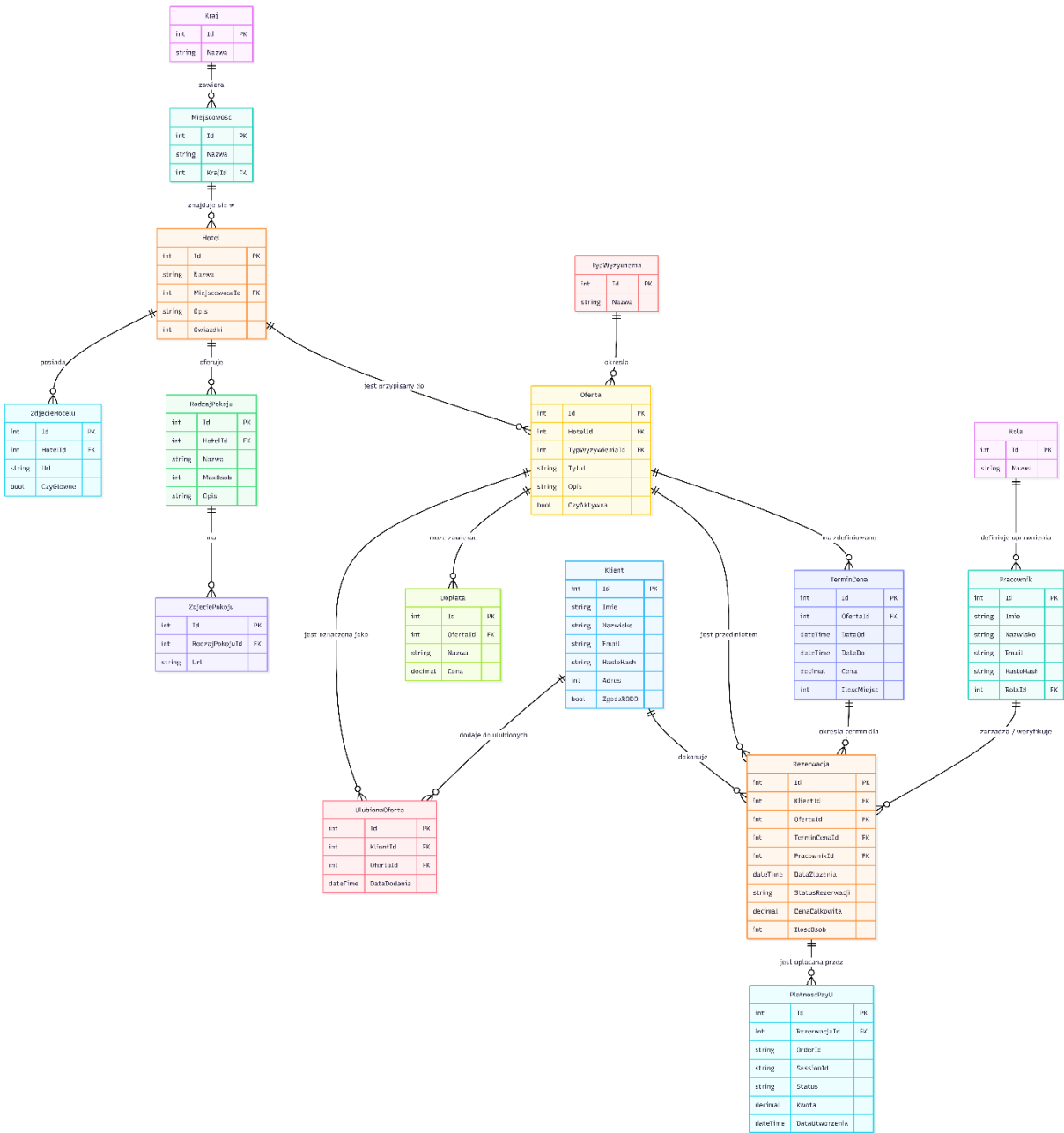
Scenariusze alternatywne

- **SCENARIUSZ (E.1):** Użytkownik podaje błędny format e-mail (brak "@" lub kropki). System dynamicznie podświetla pole na czerwono i wyświetla jasny komunikat o błędzie.
- **SCENARIUSZ (E.2):** Użytkownik próbuje wysłać formularz bez zaznaczenia zgody RODO. System blokuje proces i informuje o konieczności akceptacji zgód.

Efekt

- W bazie danych powstaje nowy rekord klienta.
- Użytkownik zostaje automatycznie zalogowany i może teraz dokonać pełnej rezerwacji wycieczki.

SPECYFIKACJA UML



INSTRUKCJA OBSŁUGI STRONY

1. Logowanie i Tworzenie Konta

- Dostęp do logowania: Na stronie głównej kliknij przycisk "Zaloguj się" w prawym górnym rogu ekranu.
- Dane logowania: Wprowadź swój adres e-mail (login) oraz hasło.
- Dane dla administratora:
 - Login: admin
 - Hasło: 123
- Zatwierdzenie: Zatwierdź proces przyciskiem "Zaloguj".

Ważne

Zaloguj się

Zarejestruj się

Logowanie

Email (Klient) lub Login (Pracownik)

admin

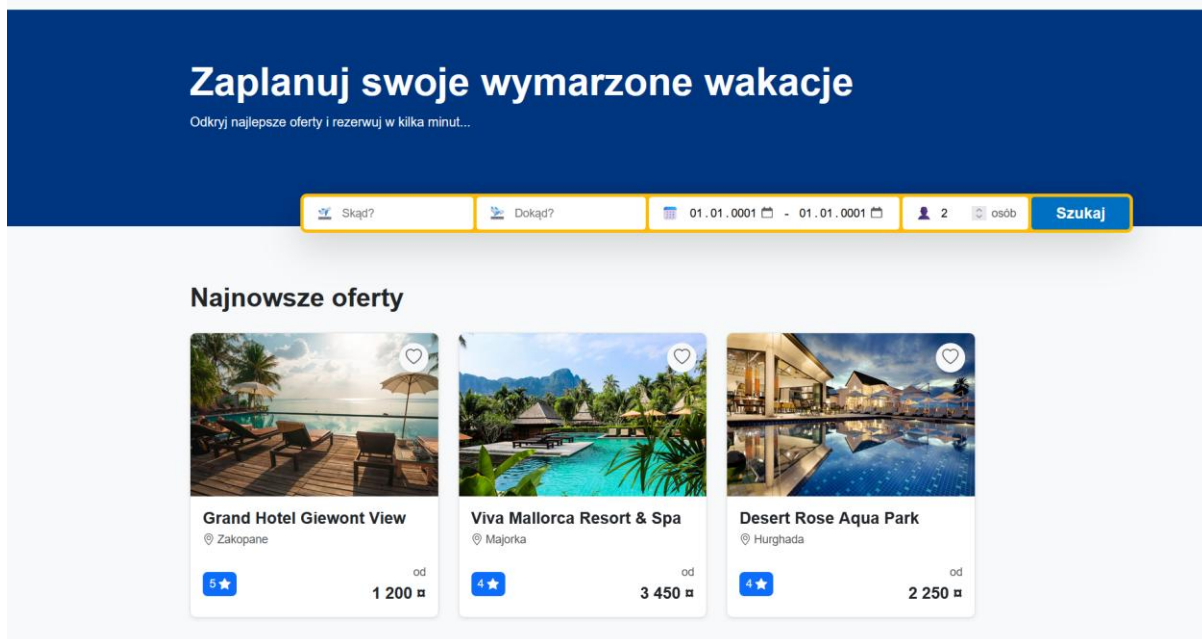
Hasło

...

Zaloguj się

2. Przeglądanie Ofert

- Strona główna: Na ekranie głównym widoczne są kafelki lub lista z dostępnymi kierunkami podróży.
- Zakres informacji: Każda pozycja zawiera podstawowe informacje: nazwę wycieczki, kraj, datę oraz cenę.
- Szczegóły: Aby poznać szczegóły, kliknij wybraną ofertę.



3. Proces Rezerwacji Wycieczki

- Konfiguracja: W widoku szczegółów wycieczki wybierz interesujący Cię termin oraz wskaż liczbę osób.
- Inicjacja rezerwacji: Kliknij przycisk "Rezerwuj" i przejdź przez następne kroki.
- Potwierdzenie: System wyświetli podsumowanie rezerwacji. Po potwierdzeniu, rezerwacja zostanie zapisana w bazie danych, a użytkownik otrzyma komunikat o sukcesie.

[Strona główna](#) / [Wyniki wyszukiwania](#) / Grand Hotel Giewont View

Grand Hotel Giewont View

Zakopane

[O ofercie](#) [Pokoje](#) [Informacje dodatkowe](#)

5★
(Kategoria hotelu)

Twoja rezerwacja

Liczba uczestników:

2

☐ Chcę wpisać własne daty

Wybierz termin pobytu:

01.07.2026 - 08.07.2026 (1 200 zł / os.)

Cena za osobę: 1 200 zł

Liczba osób: 2

Suma: 2 400 zł

REZERWUJ TERAZ

4. Panel Administratora (Admin)

- Nawigacja: Po zalogowaniu się na konto admina z lewej strony ekranu będzie dostępny dedykowany panel.
- Funkcjonalność: W panelu można przeglądać bazę danych, dodawać i edytować oferty oraz generować raporty.



Kreator Nowej Oferty

Zarządzaj i Ukrywaj Oferty

Publikacja Nowej Oferty Wakacyjnej

Wybierz zarejestrowany hotel, standard wyżywienia i wyklikaj harmonogram turnusów cenowych.

1. RELACJE SŁOWNIKOWE

Wybierz placówkę hotelową:

-- Wybierz hotel --

Standard wyżywienia:

-- Wybierz wyżywienie --

2. TERMINY WYJAZDÓW I CENNIK

+ Dodaj turnus

Brak dodanych turnusów. Kliknij przycisk powyżej, aby stworzyć termin.

Opublikuj ofertę wakacyjną

5. Panel Klienta i Zarządzanie Profilem

- Funkcje profilu: Z poziomu swojego konta możesz przeglądać własne rezerwacje oraz polubione wycieczki.
- Wylogowanie: Opcja wylogowania się z systemu jest dostępna w prawym górnym rogu ekranu.

 Ulubione

 Moje rezerwacje

Wyloguj się

SPECYFIKACJA ZEWNĘTRZNA SYSTEMU

1. Wymagania Sprzętowe i Programowe

- **System operacyjny:** Windows 10/11 (rekomendowany), macOS lub Linux.
- **Środowisko programistyczne:** Visual Studio (wersja 2022 lub nowsza) z zainstalowanym pakietem roboczym „Programowanie aplikacji internetowych i platformy ASP.NET”.
- **Framework:** .NET SDK (.net 9).
- **Serwer bazy danych:** Zainstalowany i uruchomiony serwer PostgreSQL (port **6767**).

2. Konfiguracja Bazy Danych

- **Uruchomienie:** Uruchom środowisko Visual Studio i otwórz plik rozwiązania (.sln) projektu.
- **Menedżer pakietów:** W górnym menu wybierz: Narzędzia -> Menedżer pakietów NuGet -> Konsola menedżera pakietów (Package Manager Console).
- **Inicjalizacja:** W uruchomionej konsoli wpisz następującą komendę i zatwierdź klawiszem Enter:

Bash

Update-Database

- **Automatyzacja:** Narzędzie automatycznie połączy się z serwerem PostgreSQL, utworzy bazę danych o nazwie **BookingDb** oraz wygeneruje całą strukturę tabel wraz z relacjami. Jeśli tabela z pracownikami jest pusta, system automatycznie utworzy domyślne konto administratora.

3. Konfiguracja Połączenia z Bazą Danych

- **Weryfikacja:** Przed wykonaniem migracji lub uruchomieniem systemu należy upewnić się, że aplikacja posiada poprawne dane uwierzytelniające do serwera PostgreSQL.
- **Plik konfiguracyjny:** Konfiguracja znajduje się w pliku appsettings.json w głównym katalogu projektu:

JSON

```
"ConnectionStrings": {  
  "DefaultConnection":  
    "Host=localhost;Port=6767;Database=BookingDb;Username=postgres;Password=zaq!2wsx"  
}
```


4. Konfiguracja Usługi Dev Tunnels

Charakterystyka i parametry tunelu

- **Unikalność adresu:** Każdy z deweloperów konfigurujących usługę Dev Tunnels otrzymuje swój własny, unikalny adres URL.
- **Stały adres (Static URL):** Aby zapewnić niezmiennosc wygenerowanego linku podczas kolejnych sesji i uruchomień środowiska, przy tworzeniu tunelu należy zastosować opcje **Public** (dostęp publiczny) oraz **Persistent** (adres stały).

Konfiguracja pliku appsettings.json

- **Wdrożenie adresu:** Wygenerowany adres URL stanowi bezpośredni punkt dostępowy (endpoint) do hostowanego API. Otrzymany link należy skopiować i umieścić w pliku konfiguracyjnym appsettings.json.
- **Modyfikacja portu:** W przeklejonym adresie należy ręcznie podmienić numer portu z **7048** na **7073**. Jest to krok wymagany, ponieważ docelowe API jest wystawiane właśnie na porcie **7073**.

5. Uruchomienie Aplikacji

- **Przygotowanie:** Upewnij się, że projekt jest otwarty w środowisku Visual Studio oraz że jako projekt startowy wybrana jest właściwa aplikacja webowa na górnym pasku narzędzi.
- **Uruchomienie:** Kliknij przycisk **Uruchom** (zielona strzałka z nazwą projektu lub ikona IIS Express) albo naciśnij klawisz **F5** na klawiaturze.
- **Kompilacja:** Visual Studio skompiluje kod źródłowy, pobierze niezbędne paczki NuGet i uruchomi lokalny serwer deweloperski.

6. Dostęp do Systemu

- **Adres aplikacji:** Po pomyślnym uruchomieniu, system automatycznie otworzy domyślną przeglądarkę internetową. Aplikacja systemu obsługi biura podróży jest dostępna pod adresem: <https://localhost:7048>
- **Konto testowe do weryfikacji:** W celu przetestowania pełnej funkcjonalności systemu (w tym modułu zarządzania), baza danych została automatycznie zasilona domyślnym kontem pracownika o uprawnieniach Administratora:
 - **Login (Email):** admin
 - **Hasło:** 123

SPECYFIKACJA WEWNĘTRZNA SYSTEMU

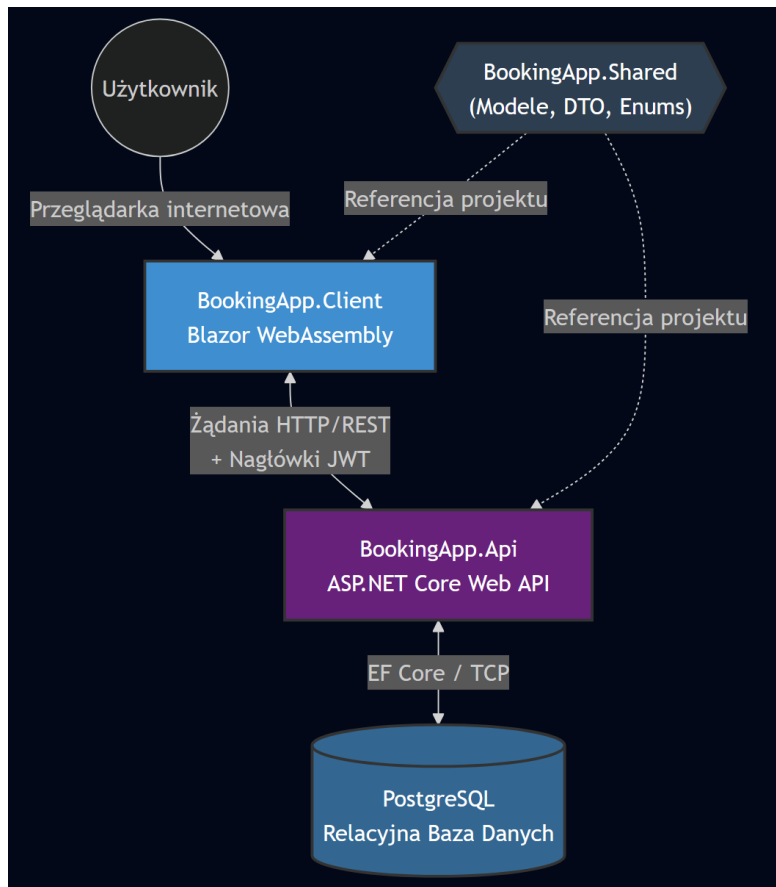
1. Architektura Systemu

System został zaprojektowany w klasycznej architekturze rozproszonej typu **Client-Server (Klient-Serwer)** i składa się z trzech wyraźnie odseparowanych od siebie projektów, co zapewnia wysoką modularność, łatwość w utrzymaniu oraz możliwość niezależnego skalowania i wdrażania poszczególnych komponentów.

Architektura opiera się na następujących modułach:

- **Aplikacja Klientcka (Frontend - Single Page Application):** Działająca w całości w przeglądarce użytkownika.
- **Interfejs Programistyczny Aplikacji (Backend - Web API):** Bezstanowy serwer obsługujący żądania biznesowe, zarządzający bazą danych i autoryzacją.
- **Biblioteka Współdzielona (Shared Library):** Centralny punkt wymiany kontraktów danych pomiędzy klientem a serwerem.

Wszystkie warstwy systemu zostały ujednolicone pod kątem platformy uruchomieniowej i korzystają z najnowszej wersji frameworka **.NET 9.0**.



2. Warstwy Aplikacji i Technologie

2.1. Warstwa Prezentacji (Frontend) – BookingApp.Client

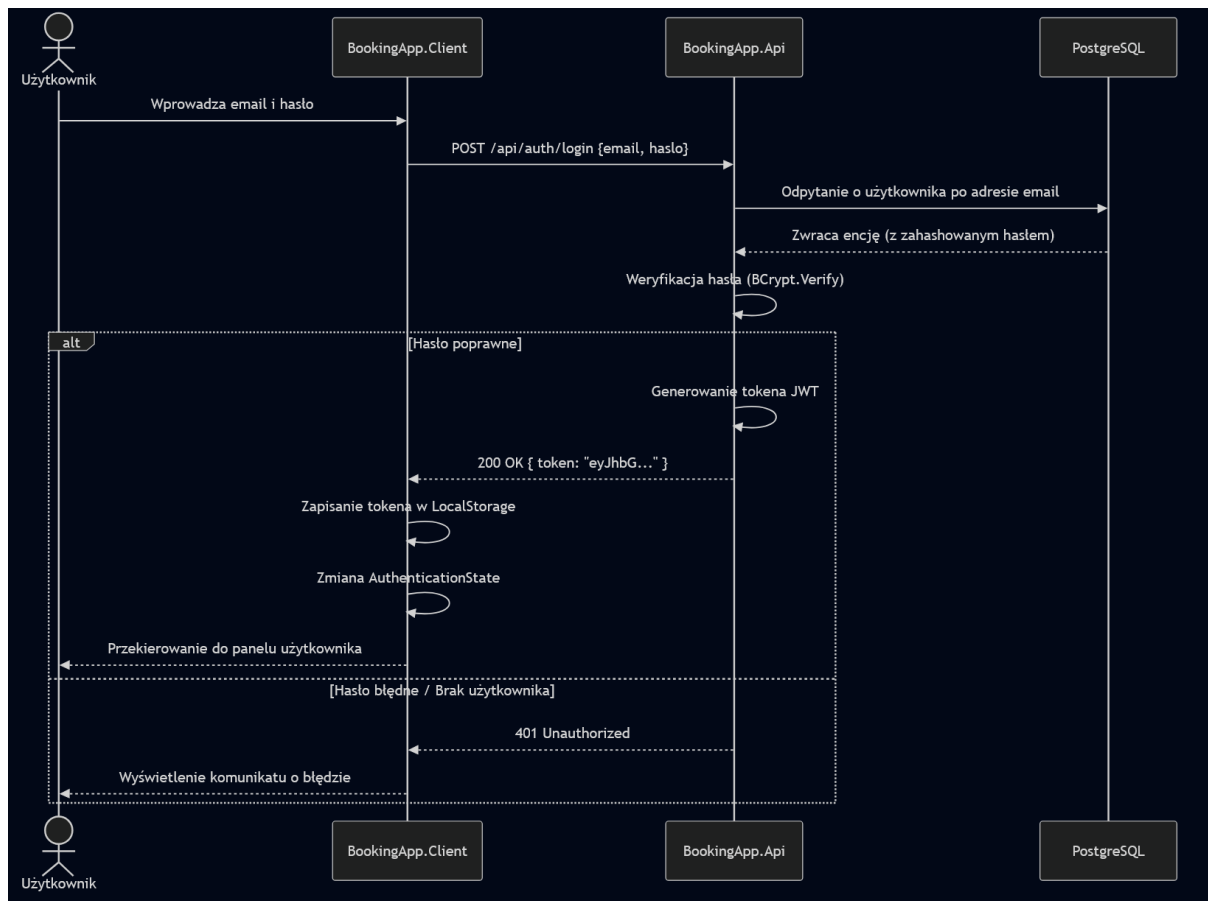
Warstwa kliencka to aplikacja typu Single Page Application (SPA) zrealizowana w technologii **Blazor WebAssembly**. Oznacza to, że kod C# oraz widoki renderowane są bezpośrednio w przeglądarce użytkownika przy użyciu technologii WebAssembly, co eliminuje konieczność ciągłego przeładowywania strony i zapewnia płynność działania zbliżoną do aplikacji desktopowych.

- **Technologia bazowa:** Microsoft.NET.Sdk.BlazorWebAssembly na platformie **.NET 9.0**.
- **Komunikacja:** Moduł Microsoft.Extensions.Http odpowiada za asynchroniczną komunikację REST API z serwerem bazowym, przysyłając zapytania zagnieżdżone wraz z nagłówkami autoryzacyjnymi.
- **Zarządzanie stanem i autoryzacja:** Implementacja Microsoft.AspNetCore.Components.Authorization pozwala na dynamiczne zarządzanie stanem uwierzytelnienia użytkownika po stronie klienta (np. ukrywanie widoków dla niezalogowanych użytkowników lub administratorów) w oparciu o tokeny.

2.2. Warstwa Logiki Biznesowej i Usług (Backend) – BookingApp.Api

Główne "serce" systemu, realizujące reguły biznesowe, walidację oraz operacje wejścia/wyjścia na danych. Aplikacja została zbudowana jako **ASP.NET Core Web API**.

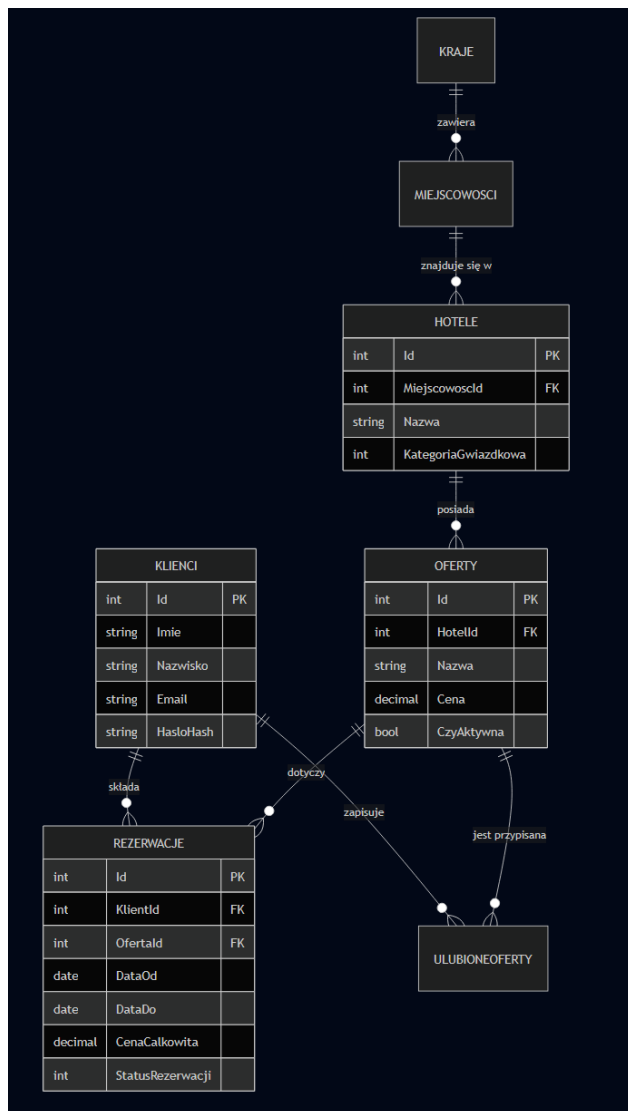
- **Technologia bazowa:** Microsoft.NET.Sdk.Web (**.NET 9.0**).
- **Bezpieczeństwo i Autoryzacja:**
 - System logowania oparty jest na tokenach **JWT (JSON Web Tokens)** implementowanych za pomocą pakietu Microsoft.AspNetCore.Authentication.JwtBearer. API działa w trybie bezstanowym.
 - Hasła użytkowników nie są przechowywane otwartym tekstem. Do ich bezpiecznego hashowania oraz weryfikacji w procesie logowania wykorzystywana jest zaawansowana biblioteka **BCrypt.Net-Next**.
- **Walidacja:** Do sprawdzania spójności i poprawności danych wejściowych z żądań HTTP zastosowano bibliotekę **FluentValidation** (wraz z rozszerzeniem do Dependency Injection). Pozwala to na oddzielenie logiki walidacyjnej od kontrolerów i utrzymanie czystości kodu.
- **Generowanie Dokumentów:** Aplikacja posiada możliwość dynamicznego generowania plików **PDF** (do raportów, potwierdzeń rezerwacji, faktur) przy użyciu wydajnej i nowoczesnej biblioteki **QuestPDF**.
- **Dokumentacja API:** Interfejsy końcowe (endpoints) są automatycznie dokumentowane zgodnie ze standardem OpenAPI/Swagger przy użyciu biblioteki **Swashbuckle.AspNetCore**.



2.3. Warstwa Dostępu do Danych (Data Access)

System komunikuje się z relacyjną bazą danych za pośrednictwem systemu mapowania obiektowo-relacyjnego (ORM).

- **Baza Danych:** Głównym silnikiem bazodanowym wykorzystywanym w projekcie jest **PostgreSQL**.
- **ORM:** Zastosowano **Entity Framework Core** w wersji 9/10 (pakiety `Microsoft.EntityFrameworkCore.Tools`, `Npgsql.EntityFrameworkCore.PostgreSQL`). Pozwala to na wykonywanie operacji na bazie danych przy użyciu języka C# i zapytań LINQ, bez konieczności pisania surowego kodu SQL.
- **Migracje:** Schemat bazy danych kontrolowany jest przez mechanizm migracji (**Code-First**) obsługiwany przez pakiet `Microsoft.EntityFrameworkCore.Design`.



2.4. Warstwa Współdzielona – BookingApp.Shared

Projekt typu **Class Library** (Microsoft.NET.Sdk), z którego korzystają zarówno API, jak i Client. Służy on jako most komunikacyjny i przechowuje kod używany po obu stronach sieci. Zazwyczaj znajdują się tutaj:

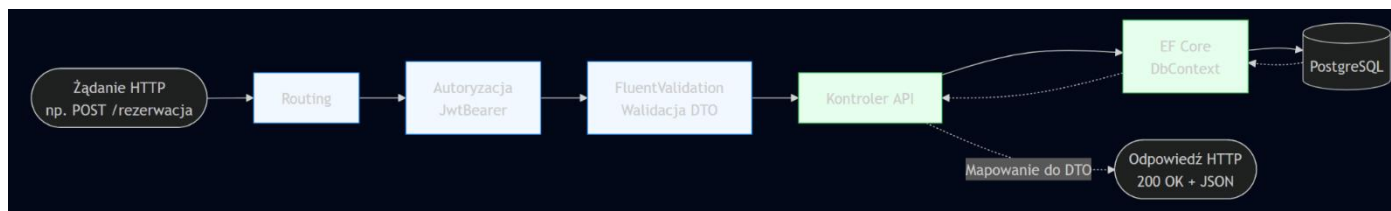
- Modele domeny.
- Obiekty **DTO (Data Transfer Objects)**.
- Enumeratory (**Enums**) używane w całym systemie.
- Interfejsy i ujednolicone komunikaty walidacyjne.

Dzięki temu zmiana struktury przesyłanych danych w jednym miejscu automatycznie rzutuje na obie części aplikacji, zmniejszając ryzyko błędów integracyjnych.

3. Zastosowane Wzorce Projektowe i Koncepcje

Biorąc pod uwagę użyte technologie w ekosystemie .NET Core, system naturalnie opiera się na najlepszych praktykach i następujących wzorcach:

- **Dependency Injection (DI - Wstrzykiwanie Zależności):** Zarówno Blazor, jak i ASP.NET Core są budowane na fundamencie wbudowanego kontenera IoC (Inversion of Control). Narzędzia takie jak FluentValidation.DependencyInjectionExtensions wskazują na intensywne korzystanie z tego wzorca. Pozwala to na wstrzykiwanie serwisów przez konstruktory, zapewniając łatwą testowalność i niski poziom sprzężenia (*low coupling*).
- **Data Transfer Object (DTO):** Poprzez wydzielenie warstwy Shared, system nie przesyła ciężkich modeli bazodanowych (Entity) bezpośrednio do klienta. Zamiast tego obiekty transformowane są do lekkich modeli DTO, które zawierają tylko te informacje, o które prosi klient.
- **Repository & Unit of Work:** Wzorec ten jest z natury zaimplementowany w samym silniku Entity Framework Core, gdzie klasa DbContext działa jako Unit of Work, a DbSet<T> pełni funkcję repozytoriów ułatwiających manipulację kolekcjami w pamięci z późniejszym zapisem do bazy Postgres.
- **Component-Based UI (Architektura komponentowa):** Narzucona przez wybór frameworka Blazor WebAssembly. Interfejs użytkownika składa się z reużywalnych, izolowanych komponentów Razor, które zawierają w sobie własną logikę oraz style.
- **Middleware Pipeline:** W ASP.NET Core API przetwarzanie żądań odbywa się poprzez rurociąg (*pipeline*). Wykorzystanie autoryzacji (JWT) czy Swaggera wskazuje na dodawanie oprogramowania pośredniczącego, które filtruje, analizuje lub zabezpiecza każde żądanie przed dotarciem do właściwych kontrolerów.



PODSUMOWANIE REALIZACJI PROJEKTU

Co Udało Się Zrealizować (Zgodnie z wymaganiami)

1. Zarządzanie bazą ofert wakacyjnych (Must Have)

- **Modele i relacje:** W systemie istnieją odpowiednie encje do pełnego zarządzania ofertą: Oferta.cs, Hotel.cs, Miejscowosc.cs, Kraj.cs, TypWyzywienia.cs, TerminCena.cs, Doplata.cs.
- **Zarządzanie statusem:** Zaimplementowano pole określające, czy oferta jest aktywna.
- **Interfejs pracownika:** Przygotowano widoki do zarządzania danymi.

2. Zarządzanie bazą klientów i autoryzacja (Must Have)

- **Modele:** Zaimplementowano profile użytkowników i klientów.
- **Rejestracja i logowanie:** Istnieje pełny moduł autoryzacji z widokami Register.razor, Login.razor, obsługiwany przez AuthController.cs i tokeny **JWT**.
- **Szyfrowanie i role:** Spełniono wymóg bezpieczeństwa – hasła są szyfrowane za pomocą biblioteki **BCrypt.Net-Next** w API, a system obsługuje role użytkowników (Rola.cs oraz powiązane handlery w Client/AuthStateProvider).

3. Wyszukiwarka i rezerwacje (Must Have / Should Have)

- **Filtry i wyszukiwanie:** Klienci mają dostęp do dedykowanego widoku Search.razor, który współpracuje z API. Budowa modeli pozwala na zaawansowane filtrowanie (po hotelu, miejscowości, wyżywieniu).
- **Obsługa rezerwacji:** Przewidziano pełny przepływ (Checkout). Zaimplementowano moduł rezerwacji (Rezerwacja.cs, RezerwacjeController.cs), widok dla klienta (MyBookings.razor) oraz zarządzanie statusami (StatusRezerwacji.cs).

4. Moduł Raportowania (Must Have / Should Have)

- **Analityka:** Projekt zawiera kontroler RaportyController.cs oraz odpowiednie modele dla różnych zestawień (FinancialReportRequestDto, PopularityReportRequestDto). Odpowiada to wymogom tworzenia raportów sprzedażowych i popularności miejscowości.
- **Eksport do zewnętrznych plików:** W projekcie BookingApp.Api.csproj zainstalowana jest biblioteka **QuestPDF**, co jednoznacznie wskazuje na realizację funkcjonalności generowania i eksportu raportów do formatu **PDF**.

Co Dodano (Funkcjonalności nadmiarowe)

- **Bramka Płatności Online (PayU):** *Miało być "Won't Have"*. Wymagania jawnie zakładały brak integracji z płatnościami online w pierwszej wersji. Tymczasem w projekcie zrealizowano pełną integrację z PayU. Świadczą o tym pliki: PayUController.cs, dedykowane modele (PayUOrderRequest.cs, PayUOrderResponse.cs), widoki frontowe (PaymentSuccess.razor, PaymentFailed.razor) oraz migracja bazy danych.
- **Moduł "Ulubione" (Wishlist):** Dodano funkcjonalność zapisywania ofert "na później", której nie było w wymaganiach bazowych. Widoczne są: encja UlubionaOferta.cs, kontroler UlubioneController.cs, widok Favorites.razor oraz odpowiadająca im migracja DB.

Czego Nie Zrealizowano

- **Integracja z zewnętrznym API pogodowym:** Brak funkcji, która automatycznie pobiera i wyświetla na karcie oferty aktualne dane pogodowe.
- **Automatyczne powiadomienia E-mail (Could Have):** Brak automatycznych powiadomień e-mail wysyłanych do klientów po zmianie statusu rezerwacji.

ZMIANY W BAZIE DANYCH

1. Integracja modułu płatności online

- **Charakterystyka zmiany:** W pierwotnych założeniach (analiza MoSCoW – status "Won't have") płatności miały być obsługiwane w pełni ręcznie.
- **Wprowadzone modyfikacje:** Aktualny schemat bazy danych został rozbudowany i pozwala na automatyczne przetrzymywanie danych transakcyjnych: ID zamówienia (OrderId), ID sesji (SessionId), kwoty oraz statusu płatności od zewnętrznego operatora.
- **Relacje:** Nowa tabela jest połączona relacją **1:N** (jeden do wielu) z tabelą Rezerwacja.

2. Rozbudowa systemu multimediiów i kategoryzacji

- **Wprowadzone modyfikacje:** Do struktury bazy danych dodano dedykowaną tabelę ZdjecieHotelu.
- **Funkcjonalność:** Umożliwia to przechowywanie wielu materiałów graficznych dla obiektów, z uwzględnieniem flagi logicznej określającej główne zdjęcie (CzyGlowne).
- **Relacje:** Tabela ZdjecieHotelu pozostaje w relacji **1:N** z tabelą Hotel.

3. Wprowadzenie funkcji "Ulubione" (Wishlist)

- **Wprowadzone modyfikacje:** Dodano nową tabelę pośredniczącą o nazwie UlubionaOferta.
- **Charakterystyka zmiany:** Rozwiązanie to pozwala na asynchroniczne zapisywanie wycieczek przez zalogowanego klienta na jego osobistej liście życzeń wraz ze stemplem czasowym (DataDodania).
- **Relacje:** Tabela realizuje relację **wiele do wielu (N:M)** pomiędzy tabelami Klient a Oferta.

4. Mechanizm miękkiego usuwania i archiwizacji (Soft Delete)

- **Wprowadzone modyfikacje:** W tabeli Oferta zaimplementowano kluczową flagę logiczną CzyAktywna (typ danych: *bool*).
- **Charakterystyka zmiany:** Zgodnie z wymaganiami funkcjonalnymi, pozwala to na archiwizację lub czasowe ukrycie wyprzedanej oferty przed klientami bez konieczności fizycznego kasowania rekordu z bazy (co mogłoby naruszyć integralność referencyjną i spójność historycznych rezerwacji).